

Lotul Restrâns de Informatică, Seniori, Ziua 1

Comisia științifică

May 12, 2026

Problema 1. Supermerge

Propusă de: Adrian Popescu

Avem N mulțimi $S_1, \dots, S_N$, inițial vide, și două operații:

- `flip(pos, val)`: inversează prezența lui val în S_{pos}
- `merge(i, j)`: $S_i, S_j \leftarrow S_i \cup S_j$
- `supermerge(st1, dr1, st2, dr2)`: pentru fiecare pereche (i, j) din cele două intervale: `merge(i, j)`

Observație importantă: Putem vedea perechea $(index, valoare)$ ca pe o celulă într-o matrice binară: `flip` = toggle pe o celulă, `merge` = aplicarea operației OR între linii. Merge nu poate transforma niciodată un bit din 1 în 0. Singurul mod prin care un bit devine 0 este prin operația de `flip`. Deci numărul total de tranziții $1 \rightarrow 0$ este cel mult Q . De aici rezultă ca numărul total de tranziții $0 \rightarrow 1$ este cel mult $N \cdot V_{\max} + Q$.

Subtask 1

Brut simplu cu `bitset` / `vector<bool>`.

Complexitate: $O(N^2 \cdot V_{\max} \cdot Q)$

Subtask 2

Optimizăm dublul for din `supermerge`.

- Pentru pozițiile $i \in [st1 + 1, dr1 - 1]$ (pozițiile *interioare* ale primului interval), candidații relevanți din intervalul $[st2, dr2]$ sunt:
 - $st2$ (capătul stâng),
 - $i - 1$, dacă $i - 1 \in [st2, dr2]$,

- $i + 1$, dacă $i + 1 \in [st2, dr2]$,
- $dr2$ (capătul drept).

Intuitiv, vecinii lui i și capetele intervalului sunt suficienți pentru a propaga toate informațiile noi.

- Pentru $i = st1$ și $i = dr1$ (capetele primului interval), parcurgem **tot** intervalul $[st2, dr2]$ cu un `for` complet.

Complexitate: $O(N \cdot V_{\max} \cdot Q)$

Subtask 3

$V_{\max} \leq 30$.

Fiecare mulțime devine o mască pe `int`.

Acum vrem să găsim rapid, păstrând ordinea din `for`-urile originale, două linii ale căror măști diferă.

Menținem pozițiile unde măștile diferă într-un set. Mai exact, păstrăm toți indicii i astfel încât:

$$mask[i] \neq mask[i - 1]$$

Vom avea două cazuri:

1. Toate elementele din intervalul $[st2, dr2]$ au aceeași mască

În acest caz, vom alege indicii din primul `for` a căror mască diferă de această mască comună.

Dacă suntem la un indice care are aceeași mască, vom sări direct la prima poziție din set mai mare decât i (unde i este indexul curent din primul `for`), deoarece este garantat că acolo vom găsi fie o valoare cu mască diferită de intervalul $[st2, dr2]$, fie se va termina bucla mare.

2. Există cel puțin două elemente diferite în intervalul $[st2, dr2]$

În acest caz, este garantat că indiferent care este indicele din primul `for`, acesta va face merge cu o linie din intervalul $[st2, dr2]$.

Dacă suntem la un indice din al doilea `for` care are aceeași mască, vom sări la următorul indice diferit folosind set-ul.

Set-ul trebuie actualizat după fiecare operație de merge, dar avem garanția că fiecare modificare relevantă din matrice poate fi găsită în:

$$O(\log N)$$

Din observația anterioară știm că numărul total de modificări reale din matrice este cel mult:

$$O(N \cdot V_{\max} + Q)$$

Rezultă că complexitatea totală este:

$$O((N \cdot V_{\max} + Q) \log N)$$

Subtask 4

supermerge doar între două poziții.

Observăm că la acest subtask nu mai este suficient să facem merge-ul naiv. Pentru a optimiza operația de merge, vom ține un arbore de intervale (*segment tree*) cu **hash-uri** în noduri (fie hash-uri polinomiale, fie XOR-hashing). Când vine o operație de merge, putem găsi toți biții care trebuie modificați în $O(\log V_{\max})$ per bit schimbat. Deoarece știm că numărul total de biți care se schimbă este mic (cel mult $N \cdot V_{\max} + Q$), complexitatea finală este:

$$O((N \cdot V_{\max} + Q) \cdot \log V_{\max})$$

Subtask 5

Combinăm for-ul optimizat de la Subtask 2 cu arborele de intervale cu hash-uri de la Subtask 4, rezultând o complexitate de:

$$O(N \cdot Q + (N \cdot V_{\max} + Q) \cdot \log V_{\max})$$

Subtask 7

Intervalele coincid.

Toate mulțimile din interval devin identice.

Algoritmul de la Subtask-ul 3 devine mai simplu, fiind suficient doar să căutăm dacă există doi indici cu hash diferit în intervalul $[st, dr]$, iar dacă există putem efectua merge-ul între ei.

$$\text{Complexitate: } O((N \cdot V_{\max} + Q) \cdot (\log N + \log V_{\max}))$$

Subtask 8

Combinăm algoritmul de la Subtask 3 cu arborele de intervale de la Subtask 4 și obținem soluția completă.

$$\text{Complexitate: } O((N \cdot V_{\max} + Q) \cdot (\log N + \log V_{\max}))$$

Problema 2. Muchii Permutate

Propusă de: Alexandru Lorintz

Soluția optimă (Complexitate $O(N \log N)$)

Problema poate fi gândită simulând procesul de construire a arborelui fix în ordinea muchiilor dată la intrare. Notăm muchiile de la 1 la $N - 1$. La un pas curent, fie acesta i , adăugăm muchia i , care va uni două componente conexe, pe care le vom nota cu A și B .

Condiția din enunț cere ca, la orice pas, ponderile muchiilor din orice componentă conexă să formeze un interval continuu de valori. Presupunând inductiv că mulțimile de ponderi pentru componentele A și B , notate $W(A)$ și $W(B)$, sunt deja intervale continue, noua mulțime de ponderi $W(A) \cup W(B) \cup \{P[i]\}$ trebuie de asemenea să formeze un interval continuu.

Deoarece cele trei entități ($W(A)$, $W(B)$ și $P[i]$) sunt blocuri distincte și independente de valori, ele pot fi așezate relativ una față de cealaltă în $3! = 6$ moduri:

1. $W(A) < W(B) < P[i]$
2. $W(B) < W(A) < P[i]$
3. $W(A) < P[i] < W(B)$
4. $W(B) < P[i] < W(A)$
5. $P[i] < W(A) < W(B)$
6. $P[i] < W(B) < W(A)$

Obiectivul problemei este minimizarea numărului de inversiuni din permutarea finală. Reamintim că o inversiune apare atunci când o muchie apărută mai devreme în input (deci cu un index mai mic) primește o pondere mai mare.

Muchia curentă i este prelucrată strict mai târziu decât toate muchiile aflate deja în componentele A și B . Cu alte cuvinte, indexul lui i este strict mai mare decât indecșii muchiilor din A și B .

- Dacă plasăm ponderea $P[i]$ la început sau la mijloc (cazurile 3, 4, 5, 6), $P[i]$ va fi strict mai mic decât ponderile unor muchii care au indecși mai mici. Acest lucru va genera garantat inversiuni suplimentare (de exemplu, $|B|$ inversiuni pentru cazul 3, sau $|A| + |B|$ inversiuni pentru cazurile 5 și 6).
- Dacă plasăm ponderea $P[i]$ la final (cazurile 1 și 2), $P[i]$ va fi maximul curent. Fiind ultima muchie adăugată și având cea mai mare pondere, ea nu va genera nicio inversiune cu muchiile din A și B .

Așadar, pentru a obține o permutare cu număr minim de inversiuni, suntem **obligați** să alegem ca $P[i]$ să fie mereu cea mai mare pondere din componentă. Cele șase variante se reduc la doar **două cazuri valide optime**: ordinea $W(A) < W(B)$ și ordinea $W(B) < W(A)$.

Dintre aceste două variante, o vom alege pe cea care generează mai puține inversiuni relativ la indecșii muchiilor din A față de cei din B . Dacă ambele variante generează același număr de inversiuni, numărul total de moduri de a construi permutarea se va înmulți cu 2, deoarece ambele ordini sunt la fel de bune.

Pentru a contoriza eficient numărul de inversiuni dintre A și B , putem menține pentru fiecare componentă conexă câte un arbore de intervale alocat dinamic, ale cărui frunze reprezintă frecvența indecșilor muchiilor conținute în acea componentă (de la 1 la $N - 1$).

La fiecare unificare a două componente, efectuăm o operație de interclasare a celor doi arbori de intervale (*Segment Tree Merge*). În timpul parcurgerii simultane a arborilor, putem calcula direct în $O(1)$ la fiecare nivel câte inversiuni s-ar forma dacă am pune A înaintea lui B (înmulțind dimensiunea subarborelui drept al lui A cu dimensiunea subarborelui stâng al lui B) și invers pentru B înaintea lui A .

Complexitatea se demonstrează standard pentru arbori de intervale alocați dinamic. Fiecare din cele $N - 1$ muchii este inserată inițial o singură dată într-un arbore gol, creând $O(\log N)$ noduri noi. Astfel, pe parcursul întregului algoritm se vor crea cel mult $O(N \log N)$ noduri în memorie. La fiecare pas de suprapunere efectuat în timpul interclasării a doi arbori, două noduri se unesc într-unul singur, reducând efectiv numărul de noduri din sistem. Deoarece putem să unim cel mult câte noduri au fost create, numărul total de pași de interclasare pe parcursul rulării este limitat superior de $O(N \log N)$.

Prin urmare, complexitatea finală a soluției (atât ca timp, cât și ca memorie) este strict $O(N \log N)$.

Problema 3. Micul String

Propusă de: Vlad-Mihai Bogdan

Pentru a calcula funcția $f(s_0, s_1, s_2, \dots, s_{K-1})$ putem aplica următorul algoritm greedy: Parcurgem de la dreapta la stânga șirurile. Când suntem la șirul s_i , avem deja generat sufixul optim obținut din șirurile $s_{i+1}, s_{i+2}, \dots, s_{K-1}$. Prin urmare, trebuie să alegem o subsecvență din s_i care adăugată în fața sufixului anterior generat să obțină șirul minim lexicografic.

Ideea principală este de a calcula $dp[p][q]$ = numărul de moduri de a construi șirurile $s_q, s_{q+1}, s_{q+2}, \dots, s_{K-1}$ astfel încât acestea să genereze sufixul $suf(p) = w_p w_{p+1} \dots w_{N-1}$. O observație importantă este că nicio subsecvență din șirul s_q nu poate fi strict mai mică lexicografic decât prefixul de aceeași lungime din $suf(p)$. Este garantat că dacă șirul s_q respectă această proprietate, atunci din el se va alege mereu un prefix de-al lui $suf(p)$.

În acest fel, pentru un p fixat, este recomandat să calculăm matricea $siruri[l][pref]$ = numărul de șiruri de lungime l în care se găsește ca subsecvență prefixul de lungime $pref$ din $suf(p)$. Această matrice se poate calcula cu ajutorul unei dinamici de tipul

$DP[l][maxpref][curpref]$ = numărul de șiruri de lungime l care se termină în starea $curpref$ dacă aplicăm algoritmul KMP pe șirul l cu pattern-ul $suf(p)$, iar starea maximă atinsă din "automatul KMP" este $maxpref$. Dinamica se poate calcula fie în $O(n^2 \cdot lmax \cdot 26)$ pentru fiecare p sau în $O(n^2 \cdot lmax)$ dacă observăm că nu este nevoie să parcurgem literele atunci când facem tranzițiile, deoarece orice literă mai mică decât cea care urmează este interzisă, respectiv orice literă mai mare ne duce înapoi în starea 0, cu o eventuală excepție la o singură poziție din $suf(p)$.

Având matricea $siruri$, putem reveni în dinamica principală, iterând după prefixul din $suf(p)$ pe care îl luăm în $siruri$, având grijă la faptul că, deși există un anumit prefix în șirul s_q tot trebuie să calculăm ce prefix vom lua în considerare să punem în față.

Complexitate finală: $O(n^3 \cdot lmax)$

Bonus: Găsiți o soluție în complexitate $O(n^2 \cdot lmax \cdot \log_2(n)^2)$.

Echipa

Problemele pentru acest baraj au fost pregătite de:

- Adrian Panaete, Colegiul Național "A.T. Laurian", Botoșani
- Nicoli Marius, Colegiul Național "Frații Buzești" Craiova
- Frâncu Cătălin, Nerdvana
- Tamio-Vesa Nakajima, Universitatea Marburg, Hesse
- Andrei-Costin Constantinescu, ETH, Zürich, Elveția
- Alexandru Lorintz, Universitatea "Babeș-Bolyai", Cluj-Napoca
- Toma Ariciu, Universitatea Politehnică București
- Andrei-Cristian Ivan, Universitatea Politehnică București
- Dan-Constantin Spătărel, București
- Vlad-Mihai Bogdan, Universitatea București
- Eugenie-Daniel Posdărăscu, Youni, București
- Radu-Teodor Prosie, Universitatea București
- Liviu Armand Gheorghe, Universitatea București
- Tiberiu Cozma, Universitatea Alexandru Ioan Cuza, Iași
- Ion Andrei Robert, Ecole Polytechnique, Paris
- Cismaru Mihaela, Google Paris
- Perju Verzotti Luca, Ecole Polytechnique, Paris
- Gheorghies Alexandru, Universitatea "Alexandru Ioan Cuza" Iași
- Boacă Andrei, Universitatea "Alexandru Ioan Cuza" Iași
- Daniel Gheorghe, Universitatea din București
- Arsenoiu Iulian, Columbia University
- Feodorov Andrei, ETH, Zürich, Elveția
- Botnaru Victor, Universitatea Politehnica Bucuresti, ACS
- Popescu Adrian, Universitatea Politehnica Bucuresti, ACS
- Măgureanu Livia, JetBrains
- Popa Bogdan-Ioan, Just 4 Kids
- Andrei Chertes, Universitatea Politehnică București